ShiftSense — Workflows and Business Rules

Version 1.0 · October 2026. All data synthetic.

This document describes every business workflow, who may act at each step, and the rules the server enforces. State tables are in `supporting/governance/workflow-state-and-decision-tables.xlsx`; diagrams are in the Markdown files under `supporting/workflows/`.

Contents

- 1. Rules that apply everywhere
- 2. Key decision rules
- Workflow: forecasting and roster planning
- Workflow: absence and recovery
- Workflow: leave review
- Workflow: attendance and corrections
- Workflow: employee onboarding
- Workflow: HR case management
- Workflow: notifications and escalations
- Workflow: data import and validation
- Workflow: authentication and account lifecycle
- Workflow: administration and audit

1. Rules that apply everywhere

- **Server-side permission** on every action (`shiftsense/services.py`): role, own team scope, account still active, no self-approval, no acting on your own HR record. Hiding a button is never the protection.
- **All-or-nothing changes** (`workflow.transaction`). A change that would add a hard-rule violation is rolled back at commit time.
- **Hard rules are never relaxed:** skills eligibility, approved leave, night permission, contract shift type, rest ≥ 11 h, maximum weekly hours, weekly off, maximum nights, extension and daily-hour limits. Contract limits can never be looser than the company-wide floor.
- **Shifts that have started are never changed** by imports, separations or recovery.
- **Reasons are mandatory** for modify, reject, escalate, revoke, decline, reset, unlock and policy changes.
- **Audit:** one append-only, hash-chained row per action.
- **Truthfulness:** an unknown value is shown as unknown (*No attendance record*, *Unavailable*, *Not configured*).

2. Key decision rules

Rule	Value
Leave coverage check	Green: team stays covered · Amber: thin · Red: short (alternatives offered)
Auto-approval of leave	Only Green, no HR trigger, and a verified sufficient balance (or unpaid)
HR leave-review triggers	unverified or insufficient balance; > 5 consecutive calendar days; sick > 2 days
Attendance lock	48 h after the shift ends; later changes only through HR correction
Check-in window	30 min before to 1 h after the shift start; once; signed-in agent only
Separation timing	Access ends at 00:00 Asia/Kolkata on the day after the last working day

Rule	Value
Absence flags	3 qualifying unplanned absences in any 30 days; **off pending validation**
Overtime approval	**Off** until a threshold is approved
Overdue alert	A critical alert undecided for 2 h (simulated) is reported to Ops once
Data alert window	After a data change: today + 1 day get alerts; later days a re-plan notice (0–3 configurable)
Coverage states	Covered 100 % · Tight 95–99.9 % · Short 85–94.9 % · Critical < 85 %
Recovery ranking	incremental cost + residual SLA risk + blast radius; minimum necessary change
Cross-skill redeployment	donor queue must stay at or above its own requirement
Swap	partner must be trained for the same queue; rules re-checked for both

Workflow: forecasting and roster planning

Owner: Analyst. Implementation: `staffing.py`, `optimizer.py`, `baseline.py`, `constraints.py`, `planning.py`.

Diagram: see the Mermaid source in documentation/supporting (renders on GitHub).

1. **Forecast.** A transparent seasonal baseline (no ML) with low/mid/high band. Labelled *prediction*.
2. **Staffing.** Erlang C per queue gives productive heads for the SLA target; shrinkage turns it into scheduled heads.
3. **Draft.** The CP-SAT optimiser (time-limited) minimises under-staffing (weighted highest), over-staffing, overtime, unfairness, preference misses and changes to the published roster. Hard rules are constraints: skills, approved leave, night permission, shift type, rest ≥ 11 h, weekly hours, weekly off, nights, extensions. Other draft sources: restore an earlier version, import a roster grid, what-if scenario.
4. **Validate.** An independent checker re-tests every hard rule (H5: zero violations).
5. **Publish.** Needs a review confirmation. Refused when the live roster changed after the draft was made (roster fingerprint), so stale drafts are never published.
6. **Benchmarks.** The manual heuristic baseline (fill highest-demand intervals first, round-robin nights) is kept for comparison on the Evidence/Benchmarks screen. All figures are simulated.

Coverage is shown on the staffing coverage matrix (queues × 30 min/1 h/2 h): Covered 100 %, Tight 95–99.9 %, Short 85–94.9 %, Critical < 85 %.

Workflow: absence and recovery

Roles: Agent / Team Lead / Ops report; Team Lead decides; Ops handles escalations. Implementation: `workflow.report_absence`, `risk.py`, `recovery.py`, `services.decide_incident`.

Diagram: see the Mermaid source in documentation/supporting (renders on GitHub).

- **Actions generated:** cross-skill redeployment (the donor queue must stay at or above its own requirement), shift swap (partner trained for the same queue), open-shift offer, shift extension / overtime.
- **Filter:** every hard rule, freeze windows, started shifts never touched.
- **Rank:** incremental cost + residual SLA risk + blast radius (people, shifts, queues touched). Minimum necessary change.
- **Residual risk is escalated**, never silently accepted. Ops decisions carry a reason; uncovered hours stay visible as carried risk.
- **Overdue:** a critical team-lead alert undecided for `recovery.tl_overdue_hours` (default 2 h) is reported to Ops once.

- The alert card shows coverage now vs after the proposed fix, marked "Proposed · not approved" until approved.

Workflow: leave review

Roles: Agent requests; HR reviews when a trigger fires; Team Lead (own team) or Ops decides. Implementation: `requests.py`, `workflow.submit_leave`, `hr.evaluate_triggers`, `services.decide_request`.

1. **Request.** Agent → Requests (form or free text such as "Sick leave on Friday"). Day from tomorrow; one open request per day.
2. **Coverage check.** Green = team stays covered; Amber = thin; Red = short, with alternatives offered (swap, other day, shift tweak).
3. **Auto-approve** only when coverage is Green, no HR trigger fires and a verified, sufficient balance exists (or the leave is unpaid). One day is debited in the same transaction.
4. **HR review triggers** (configurable, HR policy): unverified or insufficient paid balance; more than 5 consecutive calendar days; sick leave of more than 2 days. HR recommends (approve / reject / no objection / more information) with a note; tasks can be reassigned.
5. **Decision.** Team lead or Ops approves or declines (reason). Approval is refused while an HR review task is open or after the day started, and hard rules are re-checked at commit.
6. **After.** Agent can withdraw while pending; an approved request can be revoked before the day starts (shift restored, debit credited back).

Privacy. Team leads and Ops see "leave (details held by HR)": the sick-leave type and the agent's note are visible only to HR and the agent.

Workflow: attendance and corrections

Owners: Agent (check-in), Team Lead (team view), HR (corrections). Implementation: `attendance.py`, `services.check_in_self`, `hr.py`.

Source	Value stored	Notes
Agent checks in (signed in, own shift, window 30 min before to 1 h after start, once)	`checkin`	The only genuine check-in
Team lead records it	`confirmed`	via `services.correct_attendance`, inside the lock window
Attendance feed file	`feed`	Never overwrites an existing value
Demo scenario simulation	`in` / no-show	Exists only in the demo copy
Nothing recorded after the start	*No attendance record*	Unknown; never assumed absent or present

48-hour lock. 48 h after the shift ends the record locks (`attendance.lock_hours`). After that only HR can change it, through a correction request:

1. Team lead (own team) or the agent (own record) asks for a correction with a reason; one pending per shift.
2. HR (not the requester, not their own record) applies or rejects with a reason. The record must be unchanged since the request.
3. The original value is kept in history; absence counts are re-checked; requester and agent are notified.

Agents can also use *Report a record problem* (`services.report_discrepancy`), which is audited.

Workflow: employee onboarding

Owner: HR. **Supporting roles:** Admin (account), Ops (contract approval), Analyst (re-plan).

Step	Who	Where in the app	Server action / rule
1. Create the employee record	HR	HR → People data → Employees (form, Excel/CSV upload, paste, or the drop folder `data/inbox/employees/`)	`convert.py` validates every row; mandatory fields (ID, name, team, primary queue, contract) are never invented; rows with errors are held back in a rows-to-fix file
2. Skills	HR	same record: primary queue, other trained queues, skill level per queue	Eligibility is a **hard rule**: the planner and recovery never place anyone on a queue they are not trained for
3. Contract	HR proposes; Ops approves	HR → People data → Contracts	Contract limits can never be looser than the company-wide floor (`config/policy.yaml`)
4. Effective date	HR	change set preview → approve → apply	`dataflow.py`: applied on the effective date; near days raise alerts, later days produce a re-plan notice for the Analyst
5. Profile photo (optional)	HR	Employee profile → Photo	JPEG/PNG/WebP, ≤ 5 MB, resized, metadata stripped; initials when absent
6. Opening leave balance	HR, verified by a second HR user	HR → Leave & balances	Unverified balances send paid-leave requests to HR review
7. Sign-in account	Admin	Admin → People & accounts → New account	Temporary password, changed at first sign-in; or single sign-on by matching email; role always comes from ShiftSense
8. Plan	Analyst	Planning → Draft → Validate → Publish	New person is used from the next published plan

Every step writes an audit row. HR is told nothing it cannot see; the Analyst receives a notice with the effect of each applied people-data change on the plan (`services._tell_analyst`).

Workflow: HR case management

Owner: HR. Implementation: `hr.py`, `services._case_gate`, `views/hr.py`. Decision Record HR-01..HR-08.

Restricted HR cases (HR-07)

- Opened by HR with a caseworker who is an active HR user and not the subject.
- Only the assigned caseworker and Senior HR can open a case; everyone else, Admin included, is refused and the attempt is logged in `hr_case_access`.
- Entries: comments, evidence references, follow-ups with due dates. Reassign (Senior HR, reason). Close with an outcome. States: open → in progress → awaiting information → closed.

Separations (HR-01)

Record (reason category, effective date) → a **different** HR user confirms → employee "On notice" → near-day shifts become staffing alerts, later shifts are released and the Analyst is told to re-plan → at 00:00 Asia/Kolkata after the last working day the account is deactivated and the record becomes "Exited" (idempotent). Rescind with a reason reactivates the account if this separation deactivated it.

Other HR records

Record	Rule
Leave balances (HR-02)	Opening balance + source reference, verified by a second HR user; ledger of debits and credits
Review triggers (HR-03)	See leave-review.md
Attendance lock (HR-04)	48 h; corrections reviewed by HR
Absence flags (HR-05)	3 qualifying unplanned absences in any 30 days; **off pending validation**; confidential to HR
Overtime approval (HR-06)	Off until a threshold is approved
Retention (HR-08)	Deferred: no periods set, nothing deleted automatically; register shown in HR policy

HR policy is versioned; only Senior HR saves a version, approving needs a policy reference. No one acts on their own record.

Workflow: notifications and escalations

Implementation: `comms.py` (notices, messages), `attendance.py` (reminders), `mailer.py` (optional email), `jobs.py`.

Channel	Behaviour
In-app notices	Primary channel. Every role sees its own notices in the sidebar inbox
Messages	Threads between people (e.g. team lead ↔ agent)
Email (optional)	Copies of notices to a **verified** address when `SHIFTSENSE_EMAIL=smtp`. Idempotent. "Sent" means accepted by the relay, not delivered
SMS	Not used anywhere (retired)

Reminders. Check-in reminders for agents; HR reminders at most once per item per day: review tasks and correction requests waiting more than a day, case deadlines (today, tomorrow, overdue) and follow-ups (caseworker only, no case details). They run when HR opens their screens and when live time moves.

Escalations. | From | To | Trigger | |---|---|---| | Team lead alert | Ops | Team lead escalates, the approved fix leaves a gap, or the alert is overdue | | Ops | Team lead | Returned with an instruction | | Analyst risks | Ops | Planning risks flagged by the Analyst | | Governance case | Admin → Team lead / Ops | Flagged → under review → explanation requested / referred to Ops → resolved → closed | | Health check failure | `SHIFTSENSE_ALERT_EMAIL` | `health --alert`, at most once a day per failure | | People-data change | Analyst | Effect on the plan (`services._tell_analyst`) |

Notices never contain confidential HR content (absence flags are announced without names).

Workflow: data import and validation

Implementation: `convert.py` (any input), `refdata.py` (versioned records), `dataflow.py` (change engine), `schemas.py` (field definitions), `dropfolder.py`. Rules: `../data/import-validation-rules.xlsx`.

Diagram: see the Mermaid source in documentation/supporting (renders on GitHub).

- **Who owns what.** HR: employees, contracts (Ops approves), leave, balances, photos. Analyst: service levels, rules, volumes, situations, base pack. Team-lead proposals go to HR.
- **Never invented.** Mandatory fields missing → the row is held back. Bad rows go to a downloadable rows-to-fix file.

- **AI help** (`SHIFTSENSE_AI_CONVERT`): off or `nonpersonal` by default; personal data is never sent unless explicitly allowed. The AI never decides.
- **Drop folder.** `data/inbox/<kind>/`; staged by `scripts/stage_drop_folder.sh <username>`, applied by the app on refresh (only the app writes the workspace).
- **Started shifts** are never touched by any import.
- Templates: `templates/employee-import-template.csv`, `templates/call-volume-template.csv`.

Workflow: authentication and account lifecycle

Owner: Admin. Implementation: `shiftsense/security.py`, `shiftsense/auth.py`, `shiftsense/accounts.py`, `views/login.py`.

Sign-in modes (`SHIFTSENSE_AUTH`)

- `password` (default): username and password.
- `sso`: OIDC single sign-on (Microsoft Entra, Okta, Google) via Streamlit `st.login`. A person gets in only when one **active** ShiftSense account has the same email; the role is always ShiftSense's, never the identity provider's.
- `both`: either. `SHIFTSENSE_BREAKGLASS` keeps one emergency password account when SSO is the only mode.

Password rules

- Stored as PBKDF2-SHA256, 600,000 iterations, random salt per person; older hashes upgrade at the next sign-in.
- At least 12 characters, 3 of 4 character kinds, not the name or username, not a common password.
- 5 wrong attempts lock the account for 15 minutes. Idle sign-out after 30 minutes. All configurable.
- Every sign-in attempt is audited; the password never is.

Lifecycle

State / event	Who	Effect
Create	Admin	Temporary password; must be changed at first sign-in
Change own password	Anyone (sidebar → Password)	Policy enforced
Reset / unlock	Admin (reason required)	New temporary password / lock cleared; audited
Role or team change	Admin	The next action re-reads the stored account, so old permissions stop at once
Senior HR permission	Admin grants to an HR account (reason)	`accounts.extra_permissions`; counts only for the HR role
Deactivate	Admin, or automatically at 00:00 Asia/Kolkata after a confirmed separation's last working day	Next action refused; verification codes revoked
Reactivate	Admin, or rescinding the separation that deactivated it	

Demo accounts and their published passwords exist only for the demonstration. **Change or remove them before a pilot**; the health check fails in pilot mode while a demo password is still in use.

Workflow: administration and audit

Owner: Admin (oversight only: no operational decisions). Implementation: `comms.audit`, `timeline.py`, `views/admin.py`.

- **Accounts:** create, reset, unlock, deactivate, role/team change, Senior HR grant; all with reasons, all audited.
- **Governance:** scans for discrepancy signals (thresholds in `config/policy.yaml` → `governance`: approval-rate gaps between agents of one team lead, approvals that left the team short, repeated overtime, repeated swap pairs, repeated overrides); signals for review, not proof;, cases with states Flagged → Under Review → Explanation Requested / Referred to Operations → Action Required → Resolved → Closed (or Dismissed). Every scan is audited, including empty ones.
- **Audit trail:** one row per action in `audit_log` with actor, role, action, target, details. Rows are SHA-256 hash-chained and append-only (a database trigger refuses UPDATE/DELETE). `python -m shiftsense.health` verifies the chain. Audit rows are never deleted.
- **History:** `services.history` / `history_export` gives the shared event → state → recommendation → decision → outcome timeline, role-scoped.
- **Decision log:** every team-lead and Ops decision with its mandatory reason.
- **System status:** health, backups, email relay check (Admin → People & accounts).